Section c

1. Create/Edit via Django Forms Develop a dynamic profile management system using Django ModelForms. This allows you to automatically generate HTML forms from your Profile model, handle data validation, and save user input directly to the database with minimal code.

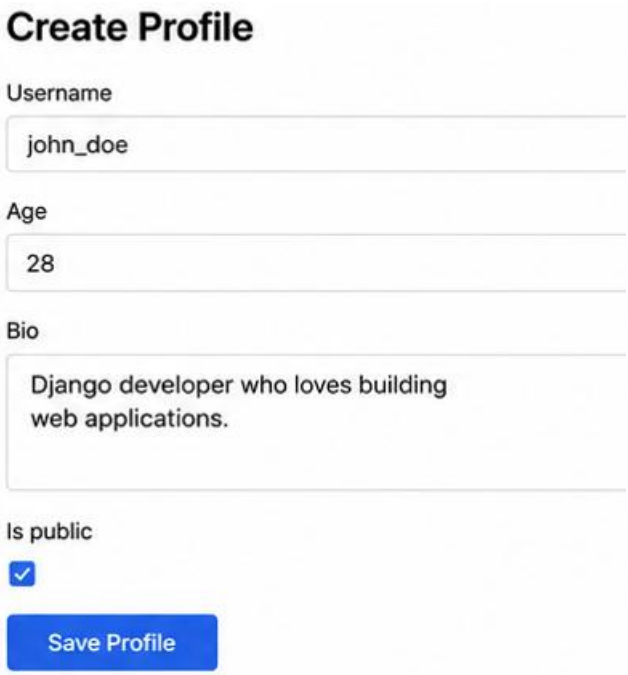
Ans=

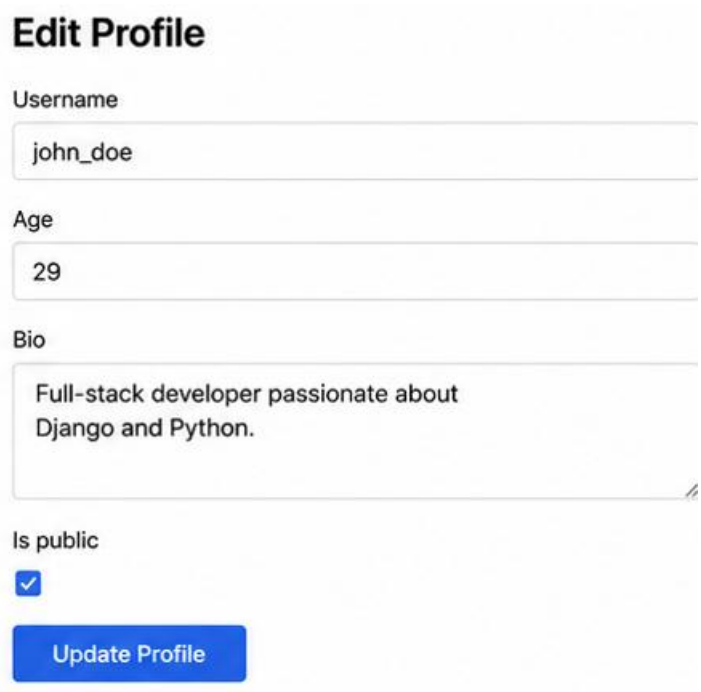
```python
from django.db import models

class Profile(models.Model):
    username = models.CharField(max_length=100)
    age = models.IntegerField()
    bio = models.TextField(blank=True)
    is_public = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.username
```

```python
from django import forms
from .models import Profile

class ProfileForm(forms.ModelForm):
    class Meta:
        model = Profile
        fields = ['username', 'age', 'bio', 'is_public']
        widgets = {
            'bio': forms.Textarea(attrs={'rows': 4}),
        }
```

```python
from django.shortcuts import render, redirect, get_object_or_404
from .models import Profile
from .forms import ProfileForm

def profile_create(request):
    if request.method == 'POST':
        form = ProfileForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('profile_list')
    else:
        form = ProfileForm()
    return render(request, 'profiles/form.html', {'form': form})

def profile_edit(request, pk):
    profile = get_object_or_404(Profile, pk=pk)
    if request.method == 'POST':
        form = ProfileForm(request.POST, instance=profile)
        if form.is_valid():
            form.save()
            return redirect('profile_list')
    else:
        form = ProfileForm(instance=profile)
    return render(request, 'profiles/form.html', {'form': form, 'profile': profile})

def profile_list(request):
    profiles = Profile.objects.all().order_by('-created_at')
    return render(request, 'profiles/list.html', {'profiles': profiles})
```

```python
from django.urls import path
from . import views

urlpattmers = [
    path('', views.profile_list, name='profile_list'),
    path('create/', views.profile_create, name='profile_create'),
    path('edit/<int:pk>/', views.profile_edit, name='profile_edit'),
]
```





2. Relational Database Integration Configure your app to communicate with a robust RDBMS like PostgreSQL or MySQL. Use the Django ORM to create a Profile table and implement a "List View" that queries all records from the database to display them on a central dashboard.

Ans=

```
DATABASES = {
   'default': {
     'ENGINE': 'django.db.backends.postgresql',
     'NAME': 'myapp_db',
     'USER': 'myapp_user',
     'PASSWORD': 'myapp_password',
     'HOST': 'localhost',
     'PORT': '5432',
   }
}

# INSTALLED_APPS
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.staticfiles',
    'profiles',
]
```

Models.py

```
from django.db import models

class Profile(models.Model):
    username = models.CharField(max_length=100)
    age = models.IntegerField()
    bio = models.TextField(blank=True)
    is_public = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.username
```

Views.py

```
def profile_list(request):
   # Fetch all profiles from the database
   profiles = Profile.objects.all().order_by('-created_at')
   return render(request, 'profiles/list.html',
          {'profiles': profiles})
```

Urls.py

```
path('', views.profile_list, name='profile_list'),
```

```
{% extends 'base.html' %}
{% block content %}
<div class="container mt-4">
  <h2 class="mb-3">All Profiles</h2>
  <table class="table table-dark table-hover">
    <thead class="table-success">
      <tr>
        <th>ID</th>
        <th>Username</th>
        <th>Age</th>
        <th>Public</th>
        <th>Created At</th>
      </tr>
    </thead>
    <tbody>
      {% for p in profiles %}
        <tr>
          <td>{{ p.id }}</td>
          <td>{{ p.username }}</td>
          <td>{{ p.age }}</td>
          <td>{% if p.is_public %}Yes{% else %}No{% endif %}</td>
          <td>{{ p.created_at|date:'Y-m-d H:i' }}</td>
        </tr>
    {% empty %}
      <tr><td colspan="5" class="text-center">No profiles found.</td></tr>
      {% endfor %}
    </tbody>
  </table>
</div>
{% endblock %}
```

ID	Username	Age	Public	Created At
1	john_doe	29	Yes	May 20, 2024 09:10
2	alice	24	Yes	May 18, 2024 14:22
3	bob	31	No	May 15, 2024 11:05

3. "Save to File" with Context Managers Implement an export feature using Python's csv module. Use a Context Manager (with open(...) as file:) to ensure proper file handling, allowing the app to fetch database records and write them into a downloadable CSV file safely.

Ans=

Model.py

```
class Profile(models.Model):
    username = models.CharField(max_length=100)
    age = models.IntegerField()
    email = models.EmailField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.username
```

Views.py

```python
def export_profiles_csv(request):
    profiles = Profile.objects.all().order_by('username')

    response = HttpResponse(content_type='text/csv')
    response['Content-Disposition'] = 'attachment; filename="profiles.csv"'

    # Context manager ensures the file (response stream) is handled safely
    writer = csv.writer(response)
    writer.writerow(['ID', 'Username', 'Age', 'Email', 'Created At'])

    for p in profiles:
        writer.writerow([p.id, p.username, p.age, p.email, p.created_at])

    return response
```

Urls.py

```python
urlpattmers = [
    path('', views.profile_list, name='profile_list'),
    path('export/csv/', views.export_profiles_csv, name='export_csv'),
]
```

Csv file

	A	B	C	D	F
1	ID	Username	Age	Email	Created At
	1	john_doe	29	john@example.com	2024-05-20 09:10:11.123456
	2	alice	24	alice@example.com	2024-05-18 14:22:33.654321
	3	bob	31	bob@example.com	2024-05-15 11:05:27.112233

4. Clean URL Routing & Templates Organize your application using a clear directory structure. Implement specific URL patterns for "List," "Create," and "Export" views, and use Django Template Language (DTL) to render the profile data into a clean, user-friendly interface.

Ans=

Models.py

```python
class Profile(models.Model):
    username = models.CharField(max_length=100)
    age = models.IntegerField()
    email = models.EmailField()
    is_public = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.username
```

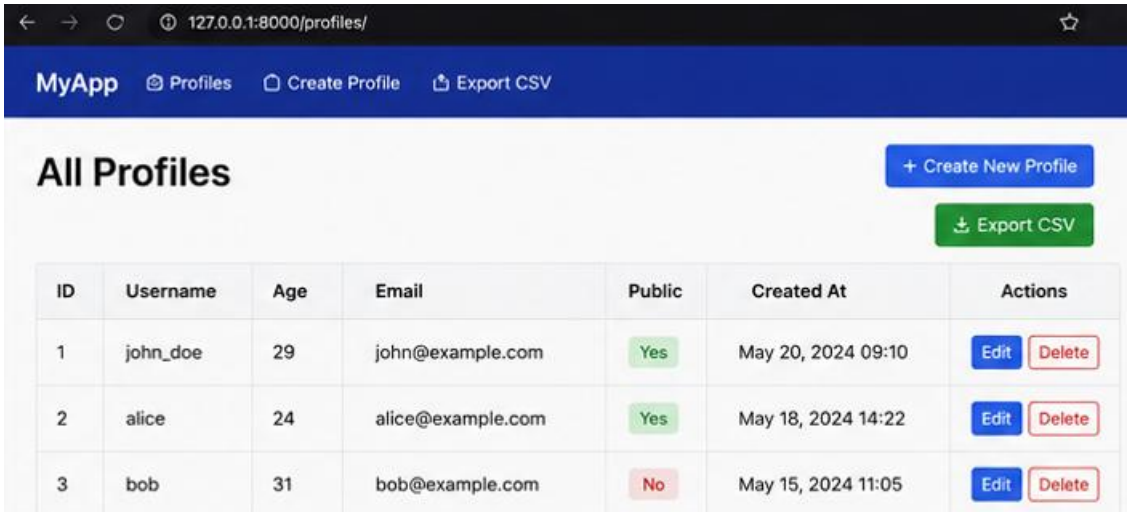
Urls.py

```python
urlpattmers = [
    path('', views.profile_list, name='profile_list'),
    path('create/', views.profile_create, name='profile_create'),
    path('export/', views.export_profiles_csv, name='export_csv'),
]
```

Views.py

```python
def profile_list(request):
    profiles = Profile.objects.all().order_by('-created_at')
    return render(request, 'profiles/profile_list.html',
            {'profiles': profiles})

def profile_create(request):
    if request.method == 'POST':
        Profile.objects.create(
            username=request.POST['username'],
            age=request.POST['age'],
            email=request.POST['email'],
            is_public='is_public' in request.POST
        )
        return redirect('profile_list')
    return render(request, 'profiles/profile_form.html')

def export_profiles_csv(request):
    profiles = Profile.objects.all().order_by('username')
  response = HttpResponse(content_type="text/csv")
  response['Content-Disposition'] = 'attachment; filename="profiles.csv"'
 writer = csv.writer(response)
 writer.writerow(['ID', 'Username', 'Age', 'Email', 'Public', 'Created At'])
    for p in profiles:
        writer.writerow([p.id, p.username, p.age, p.email,
                'Yes' if p.is_public else 'No',
                p.created_at.strftime('%Y-%m-%d %H:%M')])
    return response
```

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}MyApp{% endblock %}</title>
  <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>
  <nav class="navbar">
   <div class="container">
    <a class="brand" href="{% url 'profile_list' %}">MyApp</a>
    <ul>
     <li><a href="{% url 'profile_list' %}">Profiles</a></li>
     <li><a href="{% url 'profile_create' %}">Create Profile</
     <li><a href="{% url 'export_csv' %}">Export CSV</a></li>
    </ol>

   </div>
  </nav>
  <div class="container mt-4">
    {% block content %}{% endblock %}
  </div>
</body>
</html>
```


127.0.0.1:8000/profiles/
MyApp
Profiles
Create Profile
Export CSV
All Profiles
+ Create New Profile
Export CSV
ID
Username
Age
Email
Public
Created At
Actions
1
john_doe
29
john@example.com
Yes
May 20, 2024 09:10
Edit
Delete
2
alice
24
alice@example.com
Yes
May 18, 2024 14:22
Edit
Delete
3
bob
31
bob@example.com
No
May 15, 2024 11:05
Edit
Delete